

HTB Platform - Base de Datos

"Una base de datos bien normalizada es la diferencia entre escalar y reescribir todo desde cero."

Introducción

Este apunte documenta el diseño completo de la base de datos **HTB Platform**, un esquema relacional normalizado a **3NF** (Tercera Forma Normal) que modela una plataforma de máquinas y desafíos de hacking, similar en concepto a Hack The Box. El proyecto se desarrolló como parte del curso de introducción a bases de datos de Cisco impartido por la UNAM, con el objetivo de cumplir el requisito académico y, sobre todo, entender de primera mano cómo se diseña una base de datos relacional pensada para escalar, mantener integridad de datos. A su vez, este proyecto personal se hace con el fin de que en un futuro me ayude para que vean mis experiencias para poder trabajar en entornos laborales como Palantir.

La base de datos consta de 8 tablas y 2 vistas que calculan estadísticas y rankings en tiempo real sin duplicar información.

TABLAS (guardan datos):

1. **Categoria** – Categorías de máquinas (Linux, Web, etc.)
2. **OS** – Sistemas operativos (Linux, Windows, etc.)
3. **Dificultad** – Niveles (Fácil, Media, Difícil, Insane)
4. **Usuario** – Usuarios de la plataforma
5. **Maquina** – Máquinas/laboratorios
6. **Progreso** – Tabla asociativa: Usuario ↔ Máquina (quién completó qué)
7. **Intento** – Auditoría: cada intento de conexión VPN
8. **Comentario** – Comentarios que hacen los usuarios

VISTAS (calculan datos, no los almacenan):

1. **v_ranking_usuario_global** – Estadísticas y ranking por usuario
2. **v_ranking_maquina_dificultad** – Estadísticas por máquina

En base a lo que se impartió en el curso sobre las llaves foraneas y las terceras tablas se crearon las vistas, las vistas funcionan como una capa de metadatos sobre las tablas base, las cuales recopilan información a través de las claves foráneas (FK) para construir rankings sin necesidad de mantener tablas adicionales sincronizadas manualmente con el riesgo de una repetición de datos.

Setup inicial

Antes de crear cualquier tabla, el esquema comienza estableciendo el contexto de trabajo:

```
-- HTB Platform Database - Schema Completo E-R
```

```
CREATE DATABASE IF NOT EXISTS htb_platform;  
USE htb_platform;
```

`CREATE DATABASE` crea una base de datos nueva. La cláusula `IF NOT EXISTS` evita que el comando falle si la base ya existía, simplemente no hace nada en ese caso. Esto es importante porque permite ejecutar el script varias veces sin que falle por duplicación, un error el cual se cometió personalmente varias veces. `USE htb_platform` cambia la base de datos de trabajo para que todos los comandos siguientes operen dentro de esa base de datos. Una analogía útil con la línea de comandos de Linux, crear una base de datos es como crear una carpeta con `mkdir`, y `USE` es exactamente como hacer `cd` hacia esa carpeta antes de empezar a crear archivos dentro de ella.

Tabla 1 - Categoria

Almacena las categorías temáticas de las máquinas: Linux, Web, Windows, Active Directory, OSINT, etc.

```
CREATE TABLE Categoria (  
    id_categoria INT PRIMARY KEY AUTO_INCREMENT,  
    nombre VARCHAR(50) NOT NULL UNIQUE,  
    descripcion TEXT,  
    fecha_creacion DATETIME DEFAULT CURRENT_TIMESTAMP  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;  
  
CREATE INDEX idx_categoria_nombre ON Categoria(nombre);
```

Por qué una tabla separada?

Muchas máquinas comparten la misma categoría. Si el valor "Linux" se guardara repetido en cada fila de la tabla `Máquina`, se generaría redundancia: el mismo texto duplicado decenas o cientos de veces y esto al extraerlo a su propia tabla, "Linux" se almacena una sola vez por lo que cada máquina simplemente apunta a su id correspondiente, un ejemplo de normalización.

Columnas (o campos)

`id_categoria` es la clave primaria (`PK`), lo que garantiza que cada fila tenga un identificador único e irrepetible conocido como el `UNIQUE` y `NOT NULL` . `AUTO_INCREMENT` hace que ese identificador se asigne automáticamente y de forma incremental: la primera fila insertada recibe `id=1` , la segunda `id=2` , sin que manualmente mediante el `INSERT` se tenga que hacer, esto se asegura que este automatizado.

`nombre` es de tipo `VARCHAR(50)` (texto de hasta 50 caracteres), marcado como `NOT NULL` (obligatorio, no puede quedar vacío, sino sería anónimo) y `UNIQUE` (no puede repetirse: insertar "Linux" dos veces produciría un error y eso no puede pasar).

`descripcion` es de tipo `TEXT` , pensado para texto más largo, sin restricción de obligatoriedad.

`fecha_creacion` es de tipo `DATETIME` y usa `DEFAULT CURRENT_TIMESTAMP` : si no se especifica un valor al insertar la fila, MySQL la rellena automáticamente con la fecha y hora actuales del servidor.

La línea final define el motor de almacenamiento (`InnoDB` , que soporta relaciones y transacciones) y la codificación de caracteres(`utf8mb4` , que admite acentos, caracteres especiales e incluso emojis) esto se puso en la mayoría para que pueda tener mejor formato de caracteres y emojis.

Índices

```
CREATE INDEX idx_categoria_nombre ON Categoria(nombre);
```

Un índice funciona como el índice de un libro: permite que MySQL localice una fila concreta sin tener que revisar la tabla entera. Sin índice, buscar una categoría por nombre implicaría examinar cada fila una por una; con miles de registros, lo cual además de lento no es óptimo, por lo cual se crea un índice para datos mayores. Con el índice, MySQL puede saltar directamente al resultado.

Ejemplo de datos

<code>id_categoria</code>	<code>nombre</code>	<code>descripcion</code>
1	Linux	Máquinas Linux-based
2	Web	Desafíos Web Security
3	Windows	Máquinas Windows
4	Active Dir.	Active Directory Hacks
5	OSINT	Open Source Intel.

etc.

Tabla 2 - OS

Almacena los sistemas operativos: Linux, Windows, Web (PHP), Docker, etc.

```
CREATE TABLE OS (  
  id_os INT PRIMARY KEY AUTO_INCREMENT,  
  nombre VARCHAR(50) NOT NULL UNIQUE,  
  descripcion TEXT,  
  fecha_creacion DATETIME DEFAULT CURRENT_TIMESTAMP  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

Esta tabla es estructuralmente casi idéntica a `Categoria` : aplica la misma lógica de normalización (un OS se reutiliza entre muchas máquinas, así que se guarda una sola vez) y comparte el mismo patrón de columnas.

id_os	nombre	descripcion
1	Linux	GNU/Linux Based
2	Windows	Microsoft Windows
3	Web	Web Application
4	Docker	Containerized
5	Cloud	Cloud-based

Tabla 3 - Dificultad

Almacena los niveles de dificultad: Fácil, Media, Difícil, Insane.

```
CREATE TABLE Dificultad (  
  id_dificultad INT PRIMARY KEY AUTO_INCREMENT,  
  nombre VARCHAR(50) NOT NULL UNIQUE,  
  descripcion TEXT,  
  rango_puntos_minimo INT,  
  rango_puntos_maximo INT,  
  fecha_creacion DATETIME DEFAULT CURRENT_TIMESTAMP  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

Esta tabla añade dos columnas nuevas respecto al patrón anterior: `rango_puntos_minimo` y `rango_puntos_maximo` , que definen el rango de puntuación esperado para una máquina de

ese nivel. Por ejemplo, una máquina "Fácil" otorgaría entre 10 y 50 puntos.

id_dificultad	nombre	rango_min	rango_max
1	Fácil	10	50
2	Media	50	100
3	Difícil	100	200
4	Insane	200	500

Tabla 4 - Usuario

Almacena los usuarios registrados en la plataforma.

```
CREATE TABLE Usuario (  
  id_usuario INT PRIMARY KEY AUTO_INCREMENT,  
  nombre_usuario VARCHAR(50) NOT NULL UNIQUE,  
  apellido VARCHAR(50) NOT NULL,  
  correo VARCHAR(100) NOT NULL UNIQUE,  
  telefono VARCHAR(20),  
  fecha_nacimiento DATE,  
  pais VARCHAR(100) NOT NULL,  
  contraseña_hash VARCHAR(255) NOT NULL,  
  activo BOOLEAN NOT NULL DEFAULT TRUE,  
  fecha_registro DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  fecha_ultima_conexion DATETIME,  
  num_conexiones INT DEFAULT 0  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;  
  
CREATE INDEX idx_usuario_nombre_usuario ON Usuario(nombre_usuario);  
CREATE INDEX idx_usuario_correo ON Usuario(correo);  
CREATE INDEX idx_usuario_pais ON Usuario(pais);  
CREATE INDEX idx_usuario_activo ON Usuario(activo);
```

Algo a destacar

`nombre_usuario` y `correo` son ambas `UNIQUE`: no puede haber dos usuarios con el mismo nombre de login ni con el mismo email. Algo a recalcar es que usando principios básicos de ciberseguridad a lo largo de lo que he aprendido, se creo `contraseña_hash` para que nunca almacena la contraseña en texto plano. Se guarda como un hash, que es una secuencia de caracteres no legible, siendo esta mejor llamada una transformación criptográfica de un solo sentido, por ejemplo `abc123` se convierte en algo como `a7f3k9x2m5w8...`). Si la base de

datos llegara a filtrarse, el atacante no obtendría las contraseñas reales de los usuarios, solo sus hashes, lo cual la mayoría de las empresas de hoy en mundo, así almacenan las contraseñas por seguridad.

`pais` se marca como `NOT NULL` específicamente porque es un dato necesario para construir rankings por país, una funcionalidad que se verá en las vistas más adelante.

`activo` es de tipo `BOOLEAN` (verdadero/falso): permite desactivar una cuenta sin necesidad de eliminarla físicamente de la base de datos.

`num_conexiones` lleva un contador con `DEFAULT 0`, que se incrementa en la lógica de la aplicación cada vez que el usuario inicia sesión.

Índices

Se crean cuatro índices distintos, cada uno optimizando una búsqueda frecuente: por nombre de usuario (login), por correo (recuperación de contraseña, por ejemplo, que no debería de tener porque no es seguro eso), por país (para los rankings nacionales) y por estado activo/inactivo (para filtrar cuentas habilitadas).

id_usuario	nombre_usuario	apellido	correo	pais
1	Lobotec	García	hacker@example.com	Perú
2	securiters	López	securiters@example.com	España
3	ap4saft	Silva	saft4@example.com	Francia
4	S4vitar	Marquez	s4vitar@example.com	Marruecos

Tabla 5 - Maquina

Almacena cada máquina/laboratorio disponible en la plataforma. Es la primera tabla que introduce claves foráneas (`FK`), conectándose con las tablas de referencia ya creadas.

```
CREATE TABLE Maquina (  
  id_maquina INT PRIMARY KEY AUTO_INCREMENT,  
  nombre VARCHAR(100) NOT NULL UNIQUE,  
  descripcion TEXT,  
  puntos INT NOT NULL,  
  id_categoria INT NOT NULL,  
  id_os INT NOT NULL,  
  id_dificultad INT NOT NULL,  
  fecha_creacion DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
```

```

    activa BOOLEAN NOT NULL DEFAULT TRUE,
    num_intentos_totales INT DEFAULT 0,
    num_completadas INT DEFAULT 0,

    CONSTRAINT fk_maquina_categoria FOREIGN KEY (id_categoria)
        REFERENCES Categoria(id_categoria) ON DELETE RESTRICT ON UPDATE
    CASCADE,
    CONSTRAINT fk_maquina_os FOREIGN KEY (id_os)
        REFERENCES OS(id_os) ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_maquina_dificultad FOREIGN KEY (id_dificultad)
        REFERENCES Dificultad(id_dificultad) ON DELETE RESTRICT ON UPDATE
    CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE INDEX idx_maquina_categoria ON Maquina(id_categoria);
CREATE INDEX idx_maquina_os ON Maquina(id_os);
CREATE INDEX idx_maquina_dificultad ON Maquina(id_dificultad);
CREATE INDEX idx_maquina_nombre ON Maquina(nombre);
CREATE INDEX idx_maquina_activa ON Maquina(activa);

```

Las claves foráneas, explicadas

```

CONSTRAINT fk_maquina_categoria FOREIGN KEY (id_categoria)
    REFERENCES Categoria(id_categoria) ON DELETE RESTRICT ON UPDATE CASCADE,

```

`CONSTRAINT fk_maquina_categoria` simplemente le da un nombre a esta restricción, útil para identificarla más adelante si hay que modificarla o eliminarla. `FOREIGN KEY (id_categoria)` declara que esta columna es una clave foránea, y `REFERENCES Categoria(id_categoria)` indica a qué tabla y columna apunta.

Las dos cláusulas finales definen qué ocurre ante cambios en la tabla referenciada:

`ON DELETE RESTRICT` impide borrar una categoría si existen máquinas que la usan. Es una protección lógica: no tendría sentido eliminar "Linux" de la tabla `Categoria` si hay 50 máquinas que dependen de ese registro; MySQL rechazará el `DELETE` hasta que esas máquinas se reasignen o eliminen primero.

`ON UPDATE CASCADE` hace lo contrario para las actualizaciones: si el `id_categoria` de una categoría cambiara, todas las máquinas que la referencian se actualizarían automáticamente para mantener la consistencia, sin intervención manual.

El mismo patrón se repite para `id_os` y `id_dificultad`.

Columnas de auditoría agregada

`num_intentos_totales` y `num_completadas` son contadores que se incrementan desde la lógica de la aplicación cada vez que ocurre el evento correspondiente, evitando tener que calcular esos totales con un `COUNT()` cada vez que se necesitan.

id_maquina	nombre	puntos	id_categoria	id_dificultad
1	Lame	10	1 (Linux)	1 (Fácil)
2	Blue	30	1 (Linux)	2 (Media)
3	Devel	50	1 (Linux)	3 (Difícil)
4	Bastard	100	3 (Windows)	3 (Difícil)
5	Bashed	20	2 (Web)	1 (Fácil)

Tabla 6 - Progreso

Es la Tabla asociativa que conecta `Usuario` con `Maquina`, respondiendo a la pregunta: ¿qué máquinas ha completado cada usuario?

Por qué se necesita una tabla intermedia

Un usuario puede resolver muchas máquinas, y una máquina puede ser resuelta por muchos usuarios. Esta es una relación muchos a muchos (N:M), y el modelo relacional no permite representarla directamente entre dos tablas: se necesita una tabla intermedia que registre cada combinación válida de usuario y máquina.

```
CREATE TABLE Progreso (  
    id_usuario INT NOT NULL,  
    id_maquina INT NOT NULL,  
    fecha_inicio DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    fecha_completado DATETIME,  
    intentos INT NOT NULL DEFAULT 1,  
    estado ENUM('en_progreso', 'completado', 'abandonado') NOT NULL DEFAULT  
'en_progreso',  
    puntos_obtenidos INT,  
    tiempo_total_minutos INT,  
  
    PRIMARY KEY (id_usuario, id_maquina),  
  
    CONSTRAINT fk_progreso_usuario FOREIGN KEY (id_usuario)  
        REFERENCES Usuario(id_usuario) ON DELETE CASCADE ON UPDATE CASCADE,  
    CONSTRAINT fk_progreso_maquina FOREIGN KEY (id_maquina)
```

```
REFERENCES Maquina(id_maquina) ON DELETE CASCADE ON UPDATE CASCADE  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

```
CREATE INDEX idx_progreso_usuario ON Progreso(id_usuario);  
CREATE INDEX idx_progreso_maquina ON Progreso(id_maquina);  
CREATE INDEX idx_progreso_estado ON Progreso(estado);  
CREATE INDEX idx_progreso_fecha_completado ON Progreso(fecha_completado);
```

Clave primaria compuesta

```
PRIMARY KEY (id_usuario, id_maquina)
```

A diferencia de las tablas anteriores, **Progreso** no tiene un identificador propio tipo **id_progreso**. En su lugar, la clave primaria es la combinación de **id_usuario** e **id_maquina**. Esto significa que la pareja (usuario, máquina) debe ser única en la tabla: el mismo usuario no puede tener dos filas distintas para la misma máquina. Es exactamente el comportamiento que se necesita: un usuario tiene un único registro de progreso por máquina, sin importar cuántas veces lo intente.

El campo **estado** con **ENUM**

```
estado ENUM('en_progreso', 'completado', 'abandonado') NOT NULL DEFAULT  
'en_progreso'
```

ENUM restringe la columna a un conjunto cerrado de valores predefinidos. Aquí solo se permiten tres: **en_progreso** (el usuario está trabajando en la máquina activamente), **completado** (ya la resolvió) y **abandonado** (la dejó, por ejemplo tras 90 días sin actividad). Esto evita inconsistencias como tener "Completado", "completo" y "COMPLETADO" mezclados en la misma columna.

ON DELETE CASCADE

```
CONSTRAINT fk_progreso_usuario FOREIGN KEY (id_usuario)  
REFERENCES Usuario(id_usuario) ON DELETE CASCADE ON UPDATE CASCADE
```

Aquí la lógica es distinta a la de **Maquina**: en lugar de **RESTRICT**, se usa **CASCADE**. Si un usuario se elimina de la base de datos, todas sus filas de progreso se eliminan automáticamente junto con él. Tiene sentido: si el usuario ya no existe, sus registros de avance no tienen ningún propósito y mantenerlos sería basura huérfana en la base de datos.

id_usuario	id_maquina	fecha_inicio	estado	intentos
1	1	2025-01-15	completado	1
1	2	2025-01-20	completado	3
1	3	2025-02-01	en_progreso	5
2	1	2025-01-18	completado	2
3	4	2025-01-25	abandonado	1

Tabla 7 - Intento

Mientras que **Progreso** resume el estado final de la relación usuario-máquina, **Intento** registra cada conexión VPN individual como un evento de auditoría detallado.

```
CREATE TABLE Intento (
    id_intento INT PRIMARY KEY AUTO_INCREMENT,
    id_usuario INT NOT NULL,
    id_maquina INT NOT NULL,
    fecha_inicio DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    fecha_fin DATETIME,
    exitoso BOOLEAN NOT NULL DEFAULT FALSE,
    tipo_fallo VARCHAR(255),
    logs LONGTEXT,
    duracion_segundos INT,
    ip_conectado VARCHAR(45),

    CONSTRAINT fk_intento_usuario FOREIGN KEY (id_usuario)
        REFERENCES Usuario(id_usuario) ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_intento_maquina FOREIGN KEY (id_maquina)
        REFERENCES Maquina(id_maquina) ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE INDEX idx_intento_usuario ON Intento(id_usuario);
CREATE INDEX idx_intento_maquina ON Intento(id_maquina);
CREATE INDEX idx_intento_exitoso ON Intento(exitoso);
CREATE INDEX idx_intento_fecha_inicio ON Intento(fecha_inicio);
```

Columnas destacadas

exitoso tiene **DEFAULT FALSE**: se asume que el intento falló hasta que se confirme explícitamente lo contrario, una elección conservadora razonable para datos de auditoría.

`logs` es de tipo `LONGTEXT`, capaz de almacenar hasta 4 GB de texto, suficiente para registrar comandos ejecutados, mensajes de error o cualquier rastro útil para depuración y análisis de seguridad.

`ip_conectado` es `VARCHAR(45)`: el tamaño exacto necesario para alojar tanto direcciones IPv4 (hasta 15 caracteres, como `192.168.1.1`) como IPv6 (hasta 45 caracteres, como `2001:0db8:85a3:0000:0000:8a2e:0370:7334`).

Intento **vs** Progreso

id_intento	id_usuario	id_maquina	exitoso	tipo_fallo
1	1	1	TRUE	NULL
2	1	2	FALSE	Timeout VPN
3	1	2	FALSE	Wrong flag
4	1	2	TRUE	NULL
5	2	1	TRUE	NULL

Tabla 8 - Comentario

Almacena los comentarios que los usuarios escriben sobre máquinas que ya completaron.

```
CREATE TABLE Comentario (  
  id_comentario INT PRIMARY KEY AUTO_INCREMENT,  
  id_usuario INT NOT NULL,  
  id_maquina INT NOT NULL,  
  texto TEXT NOT NULL,  
  fecha_creacion DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  fecha_actualizacion DATETIME ON UPDATE CURRENT_TIMESTAMP,  
  
  CONSTRAINT fk_comentario_usuario FOREIGN KEY (id_usuario)  
    REFERENCES Usuario(id_usuario) ON DELETE CASCADE ON UPDATE CASCADE,  
  CONSTRAINT fk_comentario_maquina FOREIGN KEY (id_maquina)  
    REFERENCES Maquina(id_maquina) ON DELETE CASCADE ON UPDATE CASCADE,  
  CONSTRAINT fk_comentario_progreso FOREIGN KEY (id_usuario, id_maquina)  
    REFERENCES Progreso(id_usuario, id_maquina) ON DELETE CASCADE ON  
UPDATE CASCADE  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

`fecha_actualizacion` **con** `ON UPDATE`

```
fecha_actualizacion DATETIME ON UPDATE CURRENT_TIMESTAMP
```

Esta columna se actualiza automáticamente cada vez que la fila se modifica, sin que la aplicación tenga que escribir lógica adicional para registrar la fecha de edición. Es un patrón habitual para llevar trazabilidad de cambios.

La clave foránea compuesta - el detalle más importante de esta tabla

```
CONSTRAINT fk_comentario_progreso FOREIGN KEY (id_usuario, id_maquina)
    REFERENCES Progreso(id_usuario, id_maquina) ON DELETE CASCADE ON UPDATE
    CASCADE
```

Esta restricción es distinta a las dos anteriores: en lugar de referenciar una sola columna, referencia la pareja completa `(id_usuario, id_maquina)`, y exige que esa misma pareja exista previamente como fila en `Progreso`. En la práctica, esto significa que no se puede insertar un comentario para una combinación usuario-máquina que no tenga ya un registro de progreso. Es la forma en que el esquema de base de datos impone, a nivel estructural, que solo se puede comentar sobre máquinas con las que el usuario ya tiene algún historial registrado.

Vale la pena notar la limitación de este diseño: la restricción verifica que exista la pareja en `Progreso`, pero no verifica que el `estado` de esa fila sea específicamente `'completado'`. Esa validación más fina —"solo puedes comentar si ya la completaste, no si solo la intentaste"— queda como responsabilidad de la capa de aplicación, no de la base de datos.

id_comentario	id_usuario	id_maquina	texto
1	1	1	"Machine was really good"
2	2	1	"Learned a lot here"
3	1	2	"Very challenging"

Vistas analíticas

¿Qué es una vista?

Una vista (`VIEW`) es una tabla virtual: no almacena datos propios, sino que define una consulta que se ejecuta cada vez que se accede a ella. Su principal ventaja es evitar la redundancia: en lugar de mantener una tabla de "rankings" que habría que actualizar constantemente conforme

cambian los datos subyacentes, la vista recalcula el resultado en tiempo real cada vez que se consulta, garantizando que siempre refleje el estado actual de la base de datos.

Vista 1 - `v_ranking_usuario_global`

Calcula el ranking de usuarios, tanto a nivel global como dentro de su propio país.

```
CREATE OR REPLACE VIEW v_ranking_usuario_global AS
SELECT
    u.id_usuario,
    u.nombre_usuario,
    u.pais,
    u.fecha_registro,
    COALESCE(SUM(p.puntos_obtenidos), 0) AS puntos_totales,
    COUNT(CASE WHEN p.estado = 'completado' THEN 1 END) AS
maquinas_completadas,
    COUNT(CASE WHEN p.estado = 'en_progreso' THEN 1 END) AS
maquinas_en_progreso,
    COUNT(CASE WHEN p.estado = 'abandonado' THEN 1 END) AS
maquinas_abandonadas,
    ROW_NUMBER() OVER (ORDER BY COALESCE(SUM(p.puntos_obtenidos), 0) DESC) AS
posicion_global,
    ROW_NUMBER() OVER (PARTITION BY u.pais ORDER BY
COALESCE(SUM(p.puntos_obtenidos), 0) DESC) AS posicion_pais
FROM Usuario u
LEFT JOIN Progreso p ON u.id_usuario = p.id_usuario
WHERE u.activo = TRUE
GROUP BY u.id_usuario, u.nombre_usuario, u.pais, u.fecha_registro;
```

`CREATE OR REPLACE VIEW` crea la vista, y si ya existía, la reemplaza sin lanzar error, lo cual permite ejecutar el script de forma segura, como ya antes lo habíamos dicho, esto asegura que por mas nuevo que sea el usuario, pueda clonar este repo, reduciendo errores.

`COALESCE(SUM(p.puntos_obtenidos), 0)` suma todos los puntos del usuario. La función `COALESCE` resuelve un problema importante: si el usuario no tiene ninguna fila en `Progreso`, la suma daría `NULL` en lugar de `0`, y `COALESCE` sustituye ese `NULL` por un `0` explícito, evitando que el usuario desaparezca del ranking o muestre un valor confuso.

`COUNT(CASE WHEN p.estado = 'completado' THEN 1 END)` es un patrón muy usado en SQL para contar condicionalmente: el `CASE` evalúa la condición fila por fila, devuelve `1` cuando se cumple y nada (`NULL` implícito) cuando no, y `COUNT` solo cuenta los valores no nulos. El

resultado es el número de máquinas completadas por ese usuario. El mismo patrón se repite para `en_progreso` y `abandonado`

`ROW_NUMBER() OVER (ORDER BY ... DESC)` es una unción de ventana: asigna un número de fila secuencial (1, 2, 3...) según el orden especificado, en este caso de mayor a menor puntuación. Esto produce directamente la posición en el ranking global.

`ROW_NUMBER() OVER (PARTITION BY u.pais ORDER BY ... DESC)` añade `PARTITION BY`, que reinicia la numeración dentro de cada grupo. En este caso, el grupo es el país: cada país tiene su propia numeración del 1 en adelante, calculando así un ranking nacional independiente del global, en la misma consulta

`LEFT JOIN Progreso p ON u.id_usuario = p.id_usuario` es clave para que el ranking sea justo: un `LEFT JOIN` conserva todos los usuarios de la tabla `Usuario`, incluso aquellos que no tienen ninguna fila en `Progreso` (es decir, que aún no han completado ni intentado ninguna máquina). Con un `JOIN` normal (inner join), esos usuarios simplemente no aparecerían en el resultado

`WHERE u.activo = TRUE` filtra a los usuarios desactivados, que no deben estar en ningún ranking público

`GROUP BY` agrupa todas las filas de `Progreso` de un mismo usuario en una única fila de resultado, lo cual es necesario para que las funciones de agregación (`SUM`, `COUNT`) operen correctamente por usuario.

Resultado de ejemplo

id_usuario	nombre_usuario	pais	puntos_totales	posicion_global
1	hacker_mx	México	150	1
3	linux_ninja	China	120	2
2	security_pro	España	100	3
4	pwn_master	Brasil	80	4

Consultar la vista

```
-- Ranking global
SELECT * FROM v_ranking_usuario_global
ORDER BY posicion_global ASC;

-- Ranking dentro de un país específico
SELECT nombre_usuario, pais, posicion_pais, puntos_totales
```

```
FROM v_ranking_usuario_global
WHERE pais = 'México'
ORDER BY posicion_pais ASC;
```

Vista 2 - v_ranking_maquina_dificultad

EL calculo de estadísticas por máquina, cuántos usuarios la han intentado, cuántos la completaron, y el tiempo promedio que tardan, se implementó con la tabla:

```
CREATE OR REPLACE VIEW v_ranking_maquina_dificultad AS
SELECT
    m.id_maquina,
    m.nombre AS nombre_maquina,
    d.nombre AS dificultad,
    c.nombre AS categoria,
    os.nombre AS sistema_operativo,
    m.puntos,
    COUNT(DISTINCT p.id_usuario) AS intentos_totales,
    COUNT(CASE WHEN p.estado = 'completado' THEN 1 END) AS completadas,
    ROUND(100 * COUNT(CASE WHEN p.estado = 'completado' THEN 1 END) /
        NULLIF(COUNT(DISTINCT p.id_usuario), 0), 2) AS
tasa_completacion_pct,
    AVG(CASE WHEN p.estado = 'completado' THEN p.tiempo_total_minutos END) AS
tiempo_promedio_minutos
FROM Maquina m
JOIN Dificultad d ON m.id_dificultad = d.id_dificultad
JOIN Categoria c ON m.id_categoria = c.id_categoria
JOIN OS os ON m.id_os = os.id_os
LEFT JOIN Progreso p ON m.id_maquina = p.id_maquina
WHERE m.activa = TRUE
GROUP BY m.id_maquina, m.nombre, d.id_dificultad, d.nombre, c.nombre,
os.nombre, m.puntos;
```

`COUNT(DISTINCT p.id_usuario)` cuenta usuarios únicos que han interactuado con la máquina. El `DISTINCT` es necesario porque un mismo usuario podría tener múltiples filas relacionadas (en otra versión del modelo) o simplemente para garantizar que no se cuente dos veces a la misma persona.

`ROUND(100 * COUNT(...) / NULLIF(COUNT(DISTINCT p.id_usuario), 0), 2)` calcula el porcentaje de finalización: completados divididos entre intentos totales, multiplicado por 100, redondeado a 2 decimales. La función `NULLIF(valor, 0)` es la pieza defensiva: si una máquina nueva no tiene todavía ningún intento, el divisor sería `0`, y dividir por cero produce un

error en SQL. `NULLIF` convierte ese `0` en `NULL`, y dividir por `NULL` simplemente devuelve `NULL` en lugar de fallar, lo que MySQL maneja con elegancia sin interrumpir toda la consulta.

`AVG(CASE WHEN p.estado = 'completado' THEN p.tiempo_total_minutos END)` calcula el tiempo promedio solo entre las filas completadas, excluyendo intentos en progreso o abandonados, que no tendrían un tiempo de finalización significativo.

Los tres `JOIN` (no `LEFT JOIN`, sino `JOIN` normal) hacia `Dificultad`, `Categoria` y `OS` son necesarios simplemente para traer los nombres legibles de esas tablas relacionadas, en lugar de mostrar únicamente sus identificadores numéricos.

Resultado de ejemplo

nombre_maquina	dificultad	intentos_totales	tasa_completacion_pct
Lame	Fácil	150	95.33
Blue	Media	120	75.00
Devel	Difícil	80	45.00
Bastard	Difícil	50	20.00

Resumen: dónde están los rankings

Es importante recalcar que los rankings no son tablas, son vistas. No existe ninguna tabla física llamada "Ranking" que haya que actualizar manualmente; toda la lógica de cálculo vive dentro de la definición de la vista y se recalcula en cada consulta.

```
mysql htb_platform -e "SELECT * FROM v_ranking_usuario_global ORDER BY
posicion_global LIMIT 10;"
```

Próximos pasos: instalación y verificación

Instalar MariaDB en Arch Linux

```
sudo pacman -S mariadb
sudo mariadb-install-db --user=mysql --basedir=/usr --datadir=/var/lib/mysql
sudo systemctl enable mariadb
sudo systemctl start mariadb
```

Ejecutar el esquema

```
mysql -u root < 01-schema.sql
```

Verificar que todo se creó correctamente

```
mysql htb_platform -e "SHOW TABLES;"
mysql htb_platform -e "SELECT * FROM v_ranking_usuario_global;"
```

Insertar datos de prueba

```
INSERT INTO Categoria (nombre, descripcion) VALUES
('Linux', 'Máquinas basadas en Linux'),
('Web', 'Desafíos de seguridad web'),
('Windows', 'Máquinas Windows'),
('Active Directory', 'AD y directorio activo'),
('OSINT', 'Inteligencia de fuentes abiertas');

INSERT INTO Usuario (nombre_usuario, apellido, correo, pais, contraseña_hash)
VALUES
('Lobotec', 'García', 'lobotec@example.com', 'Perù', '$2b$12$...'),
('securiters', 'López', 'sec2john@example.com', 'España', '$2b$12$...'),
('S4vitar', 'Vazquez', 'marcelo@example.com', 'Marruecos', '$2b$12$...');

INSERT INTO Maquina (nombre, descripcion, puntos, id_categoria, id_os,
id_dificultad) VALUES
('Arctic', 'Samba exploit', 10, 1, 1, 1),
('Blue', 'EternalBlue', 30, 1, 2, 2),
('Markup', 'Arbitrary upload', 50, 1, 2, 3);

INSERT INTO Progreso (id_usuario, id_maquina, estado, puntos_obtenidos,
tiempo_total_minutos) VALUES
(1, 1, 'completado', 10, 45),
(1, 2, 'en_progreso', NULL, NULL),
(2, 1, 'completado', 10, 30);

INSERT INTO Intento (id_usuario, id_maquina, exitoso, ip_conectado,
duracion_segundos) VALUES
(1, 1, TRUE, '192.168.1.100', 3600);``
```